

Aprendizaje por Refuerzo

Guillermo Carmona Martínez
`g.carmonamartinez@um.es`

Francisco José Cortes Delgado
`franciscojose.cortesd@um.es`

Víctor Saura Meseguer
`victor.sauram@um.es`

12 de marzo de 2026

Resumen

En este documento se presentará uno de los trabajos que puede realizar como proyecto final y que deberán de ser expuestos en público.

Recuerde que la experimentación deberá de estar en un repositorio GitHub para poder contrastar lo que aquí explique. En el GitHub puede añadir tantos *notebooks* como considere. Pero aquí, el trabajo documental, ocupará 10 páginas descontadas la portada, el resumen, el índice y la bibliografía.

El contenido del repositorio y este documento deberán subirse al AV como evidencias del trabajo realizado.

Respete las dobles columnas. Si las suprime generará un documento que ocupará más páginas. Con las dobles columnas puede incluir más contenido.

Índice general

1. Problema del Bandido de k-brazos	2
1.1. Introducción	2
1.2. Desarrollo	2
1.3. Algoritmos	3
1.4. Evaluación/Experimentos	4
1.5. Conclusiones	6
2. Aprendizaje en Entornos Complejos	7
2.1. Introducción	7
2.2. Desarrollo	7
2.2.1. Definición Formal del Problema	7
2.2.2. Enfoques y Métodos Utilizados	8
2.3. Algoritmos	8
2.4. Evaluación/Experimentos	9
2.4.1. Configuración Experimental	9
2.4.2. Métricas	9
2.4.3. Resultados y Análisis Crítico	10
2.5. Conclusiones	11

Capítulo 1

Problema del Bandido de k -brazos

1.1. Introducción

El problema del bandido de K brazos describe un escenario en el cual un agente debe elegir repetidamente entre k opciones diferentes. Al seleccionar una acción, el agente recibe una recompensa numérica proveniente de una distribución de probabilidad desconocida (ej, normal, binomial, bernoulli, etc.) asociada a ese brazo. El objetivo fundamental es maximizar la recompensa total acumulada a lo largo de un número determinado de pasos de tiempo, lo que requiere que el agente aprenda qué acciones son las mejores basándose únicamente en la retroalimentación de sus decisiones previas.

La importancia y relevancia de este problema reside en su capacidad para modelar el dilema fundamental entre **exploración** y **explotación**. Un equilibrio inadecuado entre ambas estrategias resulta costoso, ya que centrarse solo en la explotación puede ignorar la mejor opción de forma permanente, mientras que una exploración excesiva genera un alto arrepentimiento acumulado al desperdiciar recursos en opciones subóptimas. Debido a esta capacidad para optimizar decisiones bajo incertidumbre, el modelo es muy útil y práctico en aplicaciones del mundo real, como pueden ser ensayos clínicos médicos, marketing digital (pruebas A/B), sistemas de recomendación y gestión de redes.

El objetivo de este informe es analizar y comparar las principales estrategias algorítmicas diseñadas para resolver el problema del bandido multibrazo, desde métodos básicos basados en estimaciones de valor de acción como el algoritmo del ε -greedy, hasta enfoques más sofisticados como los métodos de límite superior de confianza (UCB), el ascenso del gradiente y los métodos bayesianos. En última instancia, se busca comprender cómo estas estrategias gestionan el equilibrio exploración-explotación para maximizar la recompensa obtenida en entornos inciertos.

El informe se organiza del siguiente modo, la sección de Desarrollo presenta la definición formal del problema, revisa los enfoques existentes en la literatura y describe los métodos implementados. La sección de Algoritmos recoge todos los algoritmos implementados, explicándolos paso a paso y mostrando su pseudocódigo. La sección de Resultados recoge los experimentos realizados y analiza el comportamiento de cada algoritmo. Finalmente, la sección de Conclusiones resume los hallazgos principales.

1.2. Desarrollo

El bandido de K -brazos [?] es un proceso de decisión secuencial en el que, en cada instante $t = 1, 2, \dots, T$, un agente elige una acción $a_t \in \mathcal{A} = \{a_1, \dots, a_k\}$ y recibe una recompensa $r_t \sim p(r | a_t)$ proveniente de una distribución desconocida.

Definición 1.2.1 (Valor de una acción). El valor real de una acción a es la recompensa esperada al seleccionarla: $q(a) = \mathbb{E}[R_t | A_t = a]$.

Como $q(a)$ es desconocido, el agente debe estimarlo a partir de la experiencia. El objetivo es maximizar la recompensa acumulada $\sum_{t=1}^T r_t$, lo que equivale a minimizar el *arrepentimiento esperado*:

$$\mathbb{E}[R_T] = T q^* - \sum_{t=1}^T \mathbb{E}[r_t], \quad q^* = \max_{a \in \mathcal{A}} q(a) \quad (1.1)$$

El reto central es el equilibrio entre **exploración** (probar brazos poco conocidos para mejorar las estimaciones) y **explotación** (seleccionar el brazo con mayor recompensa esperada conocida).

Desde su formulación, han surgido cuatro grandes familias de soluciones para este problema:

1. **Métodos acción-valor con exploración heurística.** Estiman $q(a)$ como el promedio de recompensas observadas y aplican una política que intercala explotación y exploración. El ε -greedy explora al azar con probabilidad fija ε [?], mientras que el ε -decaimiento reduce ε gradualmente para favorecer la explotación conforme avanza el aprendizaje.
2. **Métodos de índice (Upper Confidence Bound, UCB).** Seleccionan la acción que maximiza un índice que combina valor estimado e incertidumbre. UCB1 [?] aplica el principio de *optimismo ante la incertidumbre* y garantiza un arrepentimiento acotado por $O(\ln T)$.
3. **Métodos de ascenso del gradiente.** Aprenden una preferencia $H_t(a)$ mediante gradiente estocástico y asignan probabilidades de selección mediante una función *softmax* [?].
4. **Métodos bayesianos.** Mantienen una distribución posterior sobre los parámetros de cada brazo, actualizada con la regla de Bayes. El **Muestreo de Thompson** [?, ?] muestrea de la posterior y selecciona el brazo con mayor valor estimado, logrando una exploración adaptativa eficaz.

También hay que mencionar una diferencia entre el aprendizaje supervisado, que proporciona **retroalimentación instructiva** (la acción correcta es conocida), y el bandido multi-brazo, el cual opera con **retroalimentación evaluativa**. En el problema del bandido el agente solo conoce la calidad de la acción tomada, no si existía una mejor [?]. Además, es un entorno **no asociativo**, pues el agente no observa ningún estado que condicione su decisión, lo que lo convierte en el modelo más simple del aprendizaje por refuerzo. A lo largo de este trabajo se asume que las distribuciones de recompensa son **estacionarias** ($p(r | a)$ no varía con el tiempo), condición bajo la que los algoritmos presentados ofrecen sus mejores garantías teóricas.

Se han implementado cinco algoritmos que cubren tres de las familias descritas anteriormente. Todos estiman los valores de acción mediante actualización incremental del promedio y se diferencian en su política de selección de brazos, **ϵ -Greedy**, que explora aleatoriamente con probabilidad fija ϵ . **ϵ -Greedy con inicialización**, variante que garantiza visitar todos los brazos antes de aplicar la política, corrigiendo el comportamiento incorrecto del greedy puro ($\epsilon = 0$). **ϵ -Decaimiento**, que reduce ϵ inversamente proporcional al tiempo para favorecer la explotación conforme avanza el aprendizaje. **UCB1** [?], que selecciona el brazo con mayor límite superior de confianza combinando valor estimado e incertidumbre. Y **Soft-max (Gibbs)**, que muestrea acciones según una distribución proporcional a sus valores estimados, controlada por una temperatura τ . Los detalles de cada algoritmo se desarrollan en profundidad en la sección de Algoritmos. Esta selección cubre una progresión natural de complejidad que permite comparar, bajo condiciones experimentales idénticas, estrategias heurísticas simples, el enfoque con garantías teóricas de UCB1 y la política probabilística continua de Softmax.

1.3. Algoritmos

A continuación se describe en detalle cada uno de los cinco algoritmos. Todos comparten la regla de actualización incremental del promedio (Algoritmo 1), que opera en tiempo y espacio $O(1)$ por paso, garantizando condiciones de comparación homogéneas. Los métodos de gradiente y bayesianos, aunque más potentes, quedan fuera del alcance de este trabajo.

Algorithm 1 Actualización incremental del valor estimado

Require: Brazo seleccionado a , recompensa observada r

- 1: $N(a) \leftarrow N(a) + 1$
 - 2: $Q(a) \leftarrow Q(a) + \frac{1}{N(a)}(r - Q(a))$
-

ϵ -Greedy

Método más simple para incorporar exploración. Con probabilidad ϵ selecciona un brazo aleatorio, con probabilidad $1 - \epsilon$ explota el brazo de mayor valor estimado. Los empates se rompen aleatoriamente para evitar sesgo hacia índices bajos. Es el punto de partida natural para entender el dilema exploración-explotación. Nótese que para $\epsilon = 0$ (greedy puro) el agente explota desde el primer paso sin haber visitado todos los brazos, limitación que corrige el algoritmo siguiente. Su simplicidad lo convierte en la línea base a implementar

ya que cualquier mejora sobre él cuantifica directamente el beneficio de estrategias más sofisticadas.

Algorithm 2 ϵ -Greedy

Require: k (brazos), $\epsilon \in [0, 1]$

- 1: $N(a) \leftarrow 0$, $Q(a) \leftarrow 0$ **para todo** a
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: **if** $\text{Uniforme}(0, 1) < \epsilon$ **then**
 - 4: $A_t \leftarrow$ brazo aleatorio de $\{1, \dots, k\}$ $\triangleright$ Exploración
 - 5: **else**
 - 6: $A_t \leftarrow \arg \max_a Q(a)$, empates rotos aleatoriamente $\triangleright$
 - 7: **end if**
 - 8: Recibir recompensa R_t
 - 9: Actualizar $N(A_t)$ y $Q(A_t)$ (Alg. 1)
 - 10: **end for**
-

ϵ -Greedy con inicialización

Extiende el anterior añadiendo una fase de arranque que visita todos los brazos al menos una vez antes de aplicar la política. Esto es imprescindible para el caso $\epsilon = 0$ (greedy puro), donde sin inicialización el agente nunca exploraría. Garantiza que $Q(a)$ se basa en al menos una observación real de cada brazo antes de tomar decisiones de explotación, haciendo la comparación con ϵ -Greedy más equitativa.

Algorithm 3 ϵ -Greedy con inicialización

Require: k , $\epsilon \in [0, 1]$

- 1: $N(a) \leftarrow 0$, $Q(a) \leftarrow 0$ **para todo** a
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: **if** $\exists a : N(a) = 0$ **then**
 - 4: $A_t \leftarrow$ brazo aleatorio de $\{a : N(a) = 0\}$ $\triangleright$
 - 5: **end if**
 - 6: Igual que Alg. 2
 - 7: **end for**
-

ϵ -Decaimiento

Reduce ϵ con el tiempo mediante decaimiento inversamente proporcional, de modo que el agente explora ampliamente al inicio y converge hacia la explotación a medida que acumula experiencia. El parámetro $\epsilon_{\min}$ evita que la exploración se anule por completo. Es el más adecuado para entornos estacionarios con T grande, donde concentrar la exploración al inicio y converger hacia la explotación es la estrategia óptima a largo plazo.

UCB1

Aplica el principio de *optimismo ante la incertidumbre*, selecciona el brazo con mayor límite superior de confianza, que combina el valor estimado con un bonus que decrece conforme el brazo es más explorado. No requiere hiperparámetro de exploración aleatorio ya que c controla únicamente la amplitud del intervalo de confianza. Es el único algoritmo del conjunto con garantías teóricas de regret sublineal ($O(k \ln T)$), lo que justifica su inclusión como cota de referencia para los métodos heurísticos.

Algorithm 4 ε -Decaimiento**Require:** $k, \varepsilon_0, \lambda > 0, \varepsilon_{\min} \geq 0$

```

1:  $N(a) \leftarrow 0, Q(a) \leftarrow 0$  para todo  $a, t \leftarrow 0$ 
2: for  $\text{paso} = 1, 2, \dots, T$  do
3:   if  $\exists a : N(a) = 0$  then
4:      $A \leftarrow$  brazo aleatorio de  $\{a : N(a) = 0\}$ 
5:   else
6:      $\varepsilon_t \leftarrow \max\left(\varepsilon_{\min}, \frac{\varepsilon_0}{1 + \lambda t}\right)$ 
7:      $A \leftarrow$  selección según Alg. 2 con  $\varepsilon = \varepsilon_t$ 
8:   end if
9:   Recibir  $R$  y actualizar  $N(A), Q(A)$ 
10: end for

```

Algorithm 5 UCB1**Require:** $k, c \geq 0$

```

1:  $N(a) \leftarrow 0, Q(a) \leftarrow 0$  para todo  $a$ 
2: for  $t = 1, 2, \dots, T$  do
3:   if  $\exists a : N(a) = 0$  then
4:      $A_t \leftarrow$  brazo aleatorio de  $\{a : N(a) = 0\}$ 
5:   else
6:      $t_{\text{total}} \leftarrow \sum_a N(a)$ 
7:      $A_t \leftarrow \arg \max_a \left[ Q(a) + c \sqrt{\frac{\ln t_{\text{total}}}{N(a)}} \right]$ 
8:   end if
9:   Recibir  $R_t$  y actualizar  $N(A_t), Q(A_t)$ 
10: end for

```

Softmax (Gibbs)

En lugar de una política binaria explorar/explotar, asigna a cada brazo una probabilidad de selección proporcional a su valor estimado mediante la función de Gibbs. La temperatura τ regula la concentración de la distribución, donde $\tau \rightarrow 0$ equivale a la política greedy, mientras que $\tau \rightarrow \infty$ produce una selección uniforme aleatoria. Para prevenir desbordamientos numéricos, se sustrae el máximo de los valores antes de calcular las exponenciales. Resulta especialmente adecuado cuando las diferencias de valor entre brazos son pequeñas, ya que la política probabilística continua evita descartar prematuramente brazos cuyas estimaciones estén próximas al óptimo.

Algorithm 6 Softmax (Gibbs)**Require:** $k, \tau > 0$

```

1:  $N(a) \leftarrow 0, Q(a) \leftarrow 0$  para todo  $a$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $Q_{\max} \leftarrow \max_a Q(a)$ 
4:    $\pi(a) \leftarrow \frac{\exp\left(\frac{Q(a) - Q_{\max}}{\tau}\right)}{\sum_{b=1}^k \exp\left(\frac{Q(b) - Q_{\max}}{\tau}\right)}$  para todo  $a$ 
5:    $A_t \leftarrow$  brazo aleatorio con probabilidades  $\pi$ 
6:   Recibir  $R_t$  y actualizar  $N(A_t), Q(A_t)$ 
7: end for

```

1.4. Evaluación/Experimentos**Configuración experimental**

Los experimentos se han implementado en Python y Jupyter Notebooks como entorno de ejecución. Se han evaluado los cinco algoritmos en **tres entornos de distribución** distintos con $k = 10$ brazos, $T = 1000$ pasos (excepto UCB1, con $T = 500$ por su convergencia más rápida) y 500 ejecuciones independientes, con semilla fija (**seed** = 42) para reproducibilidad. Los rangos de hiperparámetros explorados han sido: $\varepsilon \in \{0, 0.01, 0.1, 0.2\}$ para ε -Greedy. $\lambda \in \{0.001, 0.01, 0.1\}$ y $\varepsilon_{\min} \in \{0, 0.1\}$ para ε -Decaimiento. $c \in \{0.1, 0.5, 1, 2, 5\}$ para UCB1. $\tau \in \{0.1, 0.5, 1, 2, 5\}$ para Softmax. La comparación final entre algoritmos usa $\varepsilon=0.1$, $\lambda=0.001/\varepsilon_{\min}=0.1$, $c=1$ y $\tau=1$ en los entornos Normal y Binomial. en Bernoulli se ajustan a $c=0.5$ y $\tau=0.1$ por el comportamiento diferencial observado en ese entorno.

Adicionalmente, se realiza un estudio específico sobre el efecto de la fase de inicialización, comparando ε -Greedy estándar frente a ε -Greedy con inicialización para $\varepsilon \in \{0, 0.01, 0.1\}$ sobre el entorno Normal ($T = 1000$, 500 ejecuciones). Las distribuciones de probabilidad han sido las siguientes:

- **Normal** $\mathcal{N}(\mu, \sigma^2)$: medias μ_i muestreadas uniformemente en $[1, 10]$ con $\sigma = 1.0$. Es el entorno de referencia, con alta señal y varianza moderada.
- **Bernoulli** $\text{Bern}(p)$: probabilidades p_i muestreadas en $[0.1, 0.9]$. Recompensas binarias $\{0, 1\}$, esto conlleva mayor dificultad de discriminación por la baja varianza informativa por tirada.
- **Binomial** $\text{Bin}(n=10, p)$: probabilidades en $[0.1, 0.9]$, valor esperado $\mu_i = np_i \in [1, 9]$. Amplifica la señal Bernoulli manteniendo la misma estructura estocástica.

Métricas de evaluación

Se emplean cuatro métricas complementarias para caracterizar el comportamiento de cada algoritmo:

1. **Recompensa promedio** ($\bar{r}_t$). Media de las recompensas obtenidas en el paso t sobre todos los episodios. Refleja la velocidad de aprendizaje y el nivel de explotación alcanzado. Un algoritmo ideal converge rápidamente hacia $q^* = \max_a q(a)$.
2. **Porcentaje de selecciones óptimas** (% opt.). Fracción de pasos en que el agente elige el brazo con mayor recompensa esperada, promediada sobre episodios. Mide directamente la calidad de la política aprendida, independientemente de la escala de las recompensas.
3. **Arrepentimiento acumulado** (R_T). Diferencia entre la recompensa del agente óptimo y la del algoritmo: $R_T = T q^* - \sum_{t=1}^T r_t$. Se contrasta con la cota asintótica de Lai y Robbins [?], calculada a partir de la divergencia KL entre cada brazo subóptimo y el óptimo, que establece el crecimiento mínimo $\Theta(\ln T)$ de cualquier algoritmo consistente. Es la métrica más informativa de las empleadas, pues integra toda la historia de decisiones y permite comparar algoritmos independientemente de la escala de recompensas.
4. **Estadísticas de brazos**. Para cada algoritmo se visualiza, por brazo, la recompensa promedio observada y el número de selecciones acumuladas (indicado en la eti-

queta del eje X). Las barras se colorean distinguiendo el brazo óptimo (verde), los subóptimos (azul) y los no visitados (gris), una línea discontinua roja marca el valor de recompensa del brazo óptimo como referencia. Permite detectar si el algoritmo concentra correctamente las selecciones en el brazo óptimo o desperdicia tiradas en brazos claramente inferiores.

Resultados

ε -Greedy y ε -Greedy con inicialización

Sin inicialización, $\varepsilon=0$ produce un regret lineal (≈ 4000 a $T=1000$) al quedar atrapado en el primer brazo explotado. $\varepsilon=0.1$ es el mejor (≈ 580), aunque todos superan ampliamente la cota teórica (≈ 50). Pero al aplicar la inicialización invierte completamente el ranking, ahora $\varepsilon=0$ y $\varepsilon=0.01$ pasan a ser los mejores (≈ 145), pues identifican el óptimo durante la fase de arranque y lo explotan sin desperdicio, y $\varepsilon=0.1$ se convierte en el peor (≈ 460) porque su exploración continua genera regret innecesario una vez que todos los brazos ya han sido visitados. En conjunto, la inicialización reduce el regret máximo en un orden de magnitud.

	$\varepsilon=0$	$\varepsilon=0.01$	$\varepsilon=0.1$
Sin inicialización	4050	1340	580
Con inicialización	145	145	460

Tabla 1.1: Regret acumulado a $T=1000$ para ε -Greedy con y sin inicialización. En negrita, la mejor configuración de cada variante.

Distribución normal

UCB1 es el único algoritmo que se aproxima a la cota de Lai-Robbins ($k=10$, $T=1000$, UCB1 con $T=500$), su bono $c\sqrt{\ln t/N_i}$ dirige la exploración hacia los brazos con mayor incertidumbre, logrando regret sublineal. Softmax ($\tau=1$) es el peor (≈ 1250) porque esa temperatura aplana la distribución de selección sobre la escala $\mu_i \in [1, 10]$, comportándose como exploración casi uniforme. ε -Decaimiento produce regret lineal acotado por el piso $\varepsilon_{\min}=0.1$, incluso con λ alto. Frente a Thompson Sampling, UCB1 es más robusto al no requerir distribución a priori, aunque su rendimiento depende del ajuste de c .

Algoritmo	Mejor config.	Regret
ε -Greedy	$\varepsilon=0.1$	560
ε -Decaimiento	$\lambda=0.1$	530
UCB1	$c=1$	50[†]
Softmax	$\tau=1$	1250
Cota teórica	—	≈ 50

Tabla 1.2: Mejor configuración de cada algoritmo en distribución normal. [†]Medido a $T=500$. En negrita, el mejor resultado.

Distribución binomial

La distribución binomial ($B(10, p_i)$, $k=10$, $T=1000$, UCB1 con $T=500$) reproduce el patrón anterior con UCB1 dominante, pero la cota teórica es mayor (≈ 90) porque la KL entre

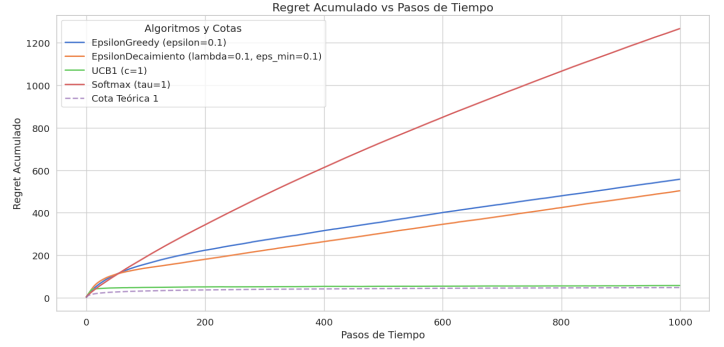


Figura 1.1: Regret acumulado en distribución normal para la mejor configuración de cada algoritmo. UCB1 ($c=1$) se aproxima a la cota teórica; el resto exhiben regret lineal o sublineal pero alejado del óptimo.

binomiales es menor que entre normales para el mismo Δ , lo que requiere más muestras para descartar brazos subóptimos. Softmax ($\tau=1$) vuelve a ser el peor, las recompensas enteras en $\{0, \dots, 10\}$ generan selección casi uniforme para esa temperatura. ε -Decaimiento mejora levemente a ε -Greedy gracias a la reducción progresiva de exploración, pero ambos exhiben regret lineal lejos del óptimo logarítmico.

Algoritmo	Mejor config.	Regret
ε -Greedy	$\varepsilon=0.1$	550
ε -Decaimiento	$\lambda=0.1$	480
UCB1	$c=1$	60[†]
Softmax	$\tau=1$	1000
Cota teórica	—	≈ 90

Tabla 1.3: Mejor configuración de cada algoritmo en distribución binomial. [†]Medido a $T=500$. En negrita, el mejor resultado.

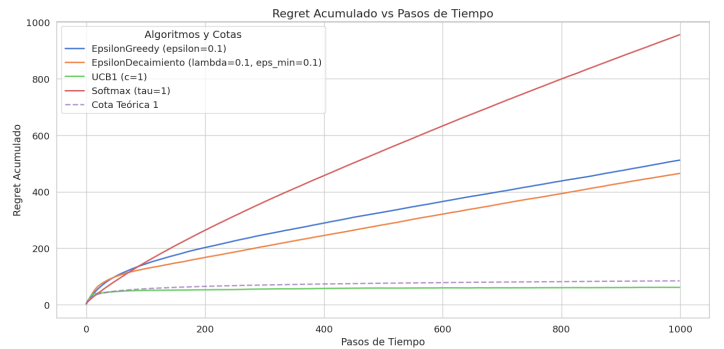


Figura 1.2: Regret acumulado en distribución binomial para la mejor configuración de cada algoritmo.

Distribución Bernoulli

La distribución Bernoulli ($Bern(p_i)$, $k=10$, $T=1000$, UCB1 con $T=500$) introduce un cambio cualitativo respecto a Normal y Binomial, al estar las recompensas en $\{0, 1\}$, los algoritmos necesitan bonos de exploración y temperaturas más pequeñas. UCB1 con $c=0.1$ logra el mejor resultado (≈ 25), muy por debajo de la cota teórica (≈ 75), mientras que Softmax invierte su ranking habitual, ahora $\tau=0.1$ es el mejor (≈ 100)

porque concentra eficazmente las selecciones en el brazo con mayor $\hat{p}$ estimada. ε -Greedy y ε -Decaimiento se aproximan a la cota teórica con sus mejores configuraciones.

Algoritmo	Mejor config.	Regret
ε -Greedy	$\varepsilon=0.1$	70
ε -Decaimiento	$\lambda=0.1$	67
UCB1	$c=0.1$	25[†]
Softmax	$\tau=0.1$	100
Cota teórica	—	≈ 85

Tabla 1.4: Mejor configuración de cada algoritmo en distribución Bernoulli. [†]Medido a $T=500$. En negrita, el mejor resultado.



Figura 1.3: Regret acumulado en distribución Bernoulli para la mejor configuración de cada algoritmo.

1.5. Conclusiones

UCB1 es el algoritmo más eficaz en las tres distribuciones estudiadas. Es el único que logra regret sublineal próximo a la cota de Lai-Robbins, gracias a que su bono de confianza $c\sqrt{\ln t/N_i}$ equilibra exploración y explotación de forma adaptativa. Su parámetro óptimo depende de la escala de recompensas, $c=1$ para Normal y Binomial, $c=0.1$ para Bernoulli, donde las recompensas en $\{0,1\}$ requieren bonos más pequeños.

La inicialización optimista transforma ε -Greedy. Sin inicialización, el ranking es $\varepsilon=0.1 \succ \varepsilon=0.01 \succ \varepsilon=0$. con inicialización, se invierte: $\varepsilon=0$ y $\varepsilon=0.01$ pasan a ser los mejores (≈ 145) porque la fase de arranque forzado ya cubre la exploración necesaria. Es una heurística eficaz pero frágil, ya que requiere conocer la escala de recompensas de antemano.

Softmax es el algoritmo más sensible a la distribución. Su temperatura óptima es $\tau=1$ en Normal y Binomial, pero $\tau=0.1$ en Bernoulli. Un valor inadecuado produce exploración casi uniforme (regret lineal elevado), lo que lo convierte en el más difícil de ajustar en la práctica.

La gráfica más relevante es el regret acumulado con cota teórica (plot_regret). Es la única que permite:

- Cuantificar el coste de oportunidad real de cada algoritmo a lo largo del tiempo.
- Comparar directamente con la cota asintótica de Lai-Robbins, determinando si el algoritmo es óptimo en or-

den de magnitud.

- Distinguir regret sublineal ($O(\ln T)$) de lineal ($O(T)$), diferencia cualitativa invisible en `plot_average_rewards`.

`plot_optimal_selections` es complementaria, confirmando si la convergencia al brazo óptimo es consistente o puntual. `plot_arm_statistics` resulta útil para diagnóstico (detectar brazos ignorados) pero no para comparar algoritmos. `plot_average_rewards` aporta la menor información diferencial, al no referenciar el óptimo teórico.

Limitaciones y líneas futuras. Los experimentos se limitan a $k=10$ y $T \leq 1000$, con un único conjunto de brazos por distribución. Ampliar a horizontes mayores y múltiples instancias aleatorias (análisis de varianza del regret) daría resultados más robustos. Como extensión natural, Thompson Sampling y KL-UCB proporcionarían cotas más ajustadas para Bernoulli y Binomial, aprovechando la estructura discreta de sus distribuciones.

Capítulo 2

Aprendizaje en Entornos Complejos

2.1. Introducción

El aprendizaje por refuerzo representa una de las ramas más diversas de la inteligencia artificial, empleada para derrotar a campeones del mundo en juegos de mesa estratégicos o en la construcción de modelos de lenguaje avanzados. A diferencia del problema del bandido de k -brazos, donde las decisiones se toman en un entorno estático, los problemas del mundo real se desarrollan en un contexto dinámico. Cada decisión tomada por el agente modifica el estado del entorno, lo que requiere cálculos constantes para actualizar su percepción del mundo.

La motivación de este trabajo radica en comprender y aplicar algoritmos capaces de resolver tareas sin un conocimiento previo de la dinámica del entorno. Se pretende evolucionar desde los métodos tabulares clásicos, viables en espacios de estados reducidos, hasta el control con aproximaciones de función mediante redes neuronales, necesario para abordar entornos continuos y complejos.

Los objetivos principales de este informe son:

1. Implementar y comparar el rendimiento de algoritmos de control tabular (Monte Carlo, SARSA y Q-Learning) en entornos de estados discretos.
2. Desarrollar y evaluar algoritmos de control aproximado (Deep Q-Learning y SARSA semi-gradiente) en entornos continuos.
3. Analizar el comportamiento, el rendimiento y la convergencia de los agentes basándose en la librería de gymnasium [3] para llevar a cabo las pruebas.

El documento se organiza de la siguiente manera: la sección de Desarrollo detalla el marco teórico necesario para aplicar los algoritmos de aprendizaje. La sección de Algoritmos expone el pseudocódigo y la lógica de los métodos implementados. Posteriormente, la Evaluación presenta los resultados empíricos obtenidos en los entornos seleccionados, y finalmente, se extraen las Conclusiones y líneas de mejora futuras.

2.2. Desarrollo

2.2.1. Definición Formal del Problema

El aprendizaje en entornos complejos se formaliza a través de un Proceso de Decisión de Markov (MDP). Un MDP se define mediante la tupla $\langle \mathcal{S}, \mathcal{A}, \{\mathcal{P}^i\}_{i=1}, \mathcal{R}, \gamma \rangle$, donde en cada instante t , el agente observa un estado $S_t \in \mathcal{S}$, ejecuta una

acción $A_t \in \mathcal{A}$ siguiendo una política $\pi(a|s)$, y transita a un nuevo estado S_{t+1} recibiendo una recompensa R_{t+1} .

Definición 2.2.1 (Política). Una política, representada como π , es una función que indica qué acción realizar de acuerdo al estado actual del entorno. Puede ser determinista, asignando una acción fija a un estado, o estocástica, proporcionando una distribución de probabilidad sobre las acciones posibles.

El objetivo del agente es encontrar la política óptima π^* que maximice el retorno esperado (la suma de recompensas descontadas a futuro):

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.1)$$

Para poder escoger la acción que lleve a un estado donde la recompensa sea máxima, el agente construye una representación interna del entorno basado en sus experiencias anteriores. La puntuación que asigna a cada estado el agente se basa en las ecuaciones de Bellman, que según el algoritmo implementado, se modifica para poder aplicar distintos enfoques. La fórmula general es la siguiente

$$v(s) = \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s] \quad (2.2)$$

Esta fórmula desglosa el valor de un estado en dos componentes principales:

- R_{t+1} : Es la recompensa inmediata que el agente recibe al realizar una transición desde el estado actual s .
- $\gamma v(S_{t+1})$: Representa el valor del siguiente estado (S_{t+1}) multiplicado por el factor de descuento γ . Este factor determina la importancia de las recompensas futuras frente a las inmediatas.

Esta ecuación representa que el valor de un estado es la recompensa inmediata más un valor a futuro, que representa el pasar a un próximo estado.

Para aplicar esta ecuación a cierto problema, se necesitan datos que involucren la obtención de recompensas.

Definición 2.2.2 (Episodio). Un episodio se define como una secuencia que involucra pasar por un estado, tomar una acción y obtener una recompensa por alcanzar otro estado, terminando el episodio en un estado terminal teóricamente. En tareas

episódicas, el agente utiliza la retroalimentación obtenida al final o durante el transcurso del episodio para maximizar la recompensa acumulativa futura

2.2.2. Enfoques y Métodos Utilizados

Cuando no se conoce el modelo de transiciones $\mathcal{P}$ ni la función de recompensa $\mathcal{R}$, el agente debe aprender a través de la experiencia. El objetivo que tiene este es actualizar su representación del entorno (tabla de estados-acciones Q) para poder elegir, dado un estado S , la acción A que maximice su retorno. Las familias de métodos estudiadas se muestran a continuación por antigüedad.

- **Métodos de Monte Carlo (MC):** Requieren que el episodio termine para calcular el retorno real G_t y actualizar el valor de los estados visitados. Son métodos de alta varianza pero sin sesgo.
- **Métodos de Diferencias Temporales (TD):** Combinan ideas de MC y Programación Dinámica. Actualizan las estimaciones basándose en otras estimaciones parciales (bootstrapping) paso a paso, sin esperar al final del episodio.
- **Control con Aproximaciones:** Cuando el espacio de estados es inmenso o continuo, es imposible mantener una tabla $Q(s, a)$. Se opta por aproximar la función de valores como una forma funcional parametrizada con un vector de pesos.

Estos enfoques mejoran gradualmente la aplicabilidad del agente. Los métodos TD mejoran a MC en entornos largos o continuos al aprender paso a paso, mientras que los métodos aproximados (DQN) suponen el salto cualitativo definitivo para resolver tareas complejas inabarcables mediante técnicas tabulares.

Aunque los últimos métodos mencionados sean los más complejos, en [2] se utilizó Q-Learning con ciertas optimizaciones para la optimización del movimiento de aeronaves en tierra dentro de los aeropuertos, demostrando ser más eficiente que los métodos tradicionales. Otro ejemplo conocido que usaba métodos aproximados es el de [1], donde se usó Deep Q-Learning para conseguir resultados propios del estado del arte en cuanto a rendimiento en juegos de la consola Atari 2600, introduciendo únicamente píxeles como input a la red.

Este trabajo se diferencia de otros enfoques previos al alejarse de la mera réplica de tutoriales para establecer y seguir un criterio de evaluación y una metodología propia.

2.3. Algoritmos

A continuación, se describen los algoritmos estudiados. Todos los agentes siguen la misma interfaz estructural basada en los métodos 'get_action(obs)' y 'update(...)' típica de gymnasium.

Monte Carlo Control (Off-policy)

Este algoritmo separa la política que se está aprendiendo (objetivo, π) de la política que genera el comportamiento (com-

portamiento o behavior, b), permitiendo aprender una política determinista óptima a partir de datos generados por una política exploratoria ε -soft. La diferencia entre las dos políticas se corrige mediante muestreo por importancia ponderado.

Algorithm 7 Control de Monte Carlo (off-policy)

```

1: Inicializar, para todo  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ :
2:    $Q(s, a) \in \mathbb{R}$  (arbitrariamente)
3:    $C(s, a) \leftarrow 0$ 
4:    $\pi(s) \leftarrow \arg \max_a Q(s, a)$  ▷ con empates rotos
   consistentemente
5: Repetir para siempre (para cada episodio):
6:    $b \leftarrow$  cualquier política blanda (soft policy)
7:   Generar un episodio usando  $b$ :
      $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$ 
8:    $G \leftarrow 0$ 
9:    $W \leftarrow 1$ 
10:  for  $t = T - 1, T - 2, \dots, 0$  do
11:     $G \leftarrow \gamma G + R_{t+1}$ 
12:     $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$ 
13:     $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$ 
14:     $\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$  ▷ con empates rotos
    consistentemente
15:    if  $A_t \neq \pi(S_t)$  then
16:      salir del bucle interno ▷ proceder al siguiente
    episodio
17:    end if
18:     $W \leftarrow W \frac{1}{b(A_t|S_t)}$ 
19:  end for

```

A pesar de que en la práctica los algoritmos de diferencias temporales suelen ser más eficientes, el concepto de Monte Carlo se aplica en algoritmos más complejos como REINFORCE que se basa en el gradiente de política.

Monte Carlo Control First-Visit (On-policy)

Este algoritmo es un caso concreto del anterior, donde la política de comportamiento y la política objetivo son la misma. Además solo se actualiza un par (estado, valor) la primera vez que aparece en un episodio.

Sarsa (on-policy TD control)

A diferencia de los métodos de Monte Carlo, Sarsa es un algoritmo de Diferencias Temporales (TD) que actualiza la tabla de valores Q cada paso (con TD(0)). Es un método on-policy ya que utiliza la misma política para obtener la próxima acción del agente, necesaria para calcular el valor esperado del estado actual.

SARSA es un algoritmo que toma en cuenta sus decisiones en el futuro, por lo que su política se vuelve más conservadora.

Q-Learning (off-policy TD control)

Este es un método off-policy que actualiza $Q(S, A)$ asumiendo que en el siguiente estado se tomará la acción óptima ($\max_a Q(S', a)$), incluso si el agente termina realizando una acción diferente por exploración. Q-Learning, al ser off-policy, se aproxima más a la política óptima que SARSA debido a

Algorithm 8 Sarsa: Control TD en la política (on-policy) para estimar $Q \approx q_*$

Require: Parámetros del algoritmo: tamaño de paso $\alpha \in (0, 1]$, $\varepsilon > 0$ pequeño

- 1: Inicializar $Q(s, a)$, para todo $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrariamente excepto que $Q(\text{terminal}, \cdot) = 0$
- 2: **for** cada episodio **do**
- 3: Inicializar S
- 4: Elegir A desde S usando una política derivada de Q (p.e., ε -greedy)
- 5: **while** S no sea terminal **do**
- 6: Realizar la acción A , observar R y el siguiente estado S'
- 7: Elegir A' desde S' usando una política derivada de Q (p.e., ε -greedy)
- 8: $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$
- 9: $S \leftarrow S'$
- 10: $A \leftarrow A'$
- 11: **end while**
- 12: **end for**

Algorithm 9 Q-learning: Control TD fuera de la política (off-policy) para estimar $\pi \approx \pi_*$

Require: Parámetros del algoritmo: tamaño de paso $\alpha \in (0, 1]$, $\varepsilon > 0$ pequeño

- 1: Inicializar $Q(s, a)$, para todo $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrariamente excepto que $Q(\text{terminal}, \cdot) = 0$
- 2: **for** cada episodio **do**
- 3: Inicializar S
- 4: **repeat** ▷ Para cada paso del episodio
- 5: Elegir A desde S usando una política derivada de Q (p.e., ε -greedy)
- 6: Realizar la acción A , observar R y el siguiente estado S'
- 7: $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
- 8: $S \leftarrow S'$
- 9: **until** S es terminal
- 10: **end for**

que su política objetivo es greedy y siempre escoge el estado con máximo valor.

Sarsa semi-gradiente episódico

A diferencia de los métodos tabulares, SARSA semi-gradiente utiliza una función de valor de acción parametrizada ($\hat{q}$) y actualiza un vector de pesos ($\mathbf{w}$) mediante descenso de gradiente para generalizar el aprendizaje en espacios de estados grandes o continuos.

Este algoritmo permite manejar espacios de estado enormes o continuos al no depender de una tabla de valores Q .

2.4. Evaluación/Experimentos

2.4.1. Configuración Experimental

Las pruebas se realizaron con la librería Gymnasium y semilla fija (SEED=2024) para garantizar reproducibilidad. Se

Algorithm 10 Sarsa semi-gradiente episódico para estimar $\hat{q} \approx q_*$

1: **Entrada:** una parametrización de función de valor de acción diferenciable $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Require: tamaño de paso $\alpha > 0$, $\varepsilon > 0$ pequeño

- 2: Inicializar los pesos de la función de valor $\mathbf{w} \in \mathbb{R}^d$ arbitrariamente (p.e., $\mathbf{w} = \mathbf{0}$)
- 3: **for** cada episodio **do**
- 4: $S, A \leftarrow$ estado y acción inicial del episodio (p.e., ε -greedy)
- 5: **while** S no sea terminal **do**
- 6: Realizar la acción A , observar R y el siguiente estado S'
- 7: **if** S' es terminal **then**
- 8: $\mathbf{w} \leftarrow \mathbf{w} + \alpha[R - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$
- 9: **ir al siguiente episodio**
- 10: **end if**
- 11: Elegir A' como una función de $\hat{q}(S', \cdot, \mathbf{w})$ (p.e., ε -greedy)
- 12: $\mathbf{w} \leftarrow \mathbf{w} + \alpha[R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$
- 13: $S \leftarrow S'$
- 14: $A \leftarrow A'$
- 15: **end while**
- 16: **end for**

estudiaron tres entornos representativos:

- ▶ **Taxi-v3 (discreto):** 500 estados, 6 acciones. Recompensas: -1 por paso, $+20$ entrega exitosa, -10 acción ilegal. Entrenamiento de 100 000 episodios para comparar MC, SARSA y Q-Learning.
- ▶ **CliffWalking-v1 (discreto):** grid 4×12 (48 estados), 4 acciones. Recompensas: -1 por paso, -100 caída al acantilado y 0 al llegar. Entrenamiento de 10 000 episodios para comparar Q-Learning vs SARSA.
- ▶ **LunarLander-v3 (continuo):** estado de 8 variables continuas y 4 acciones. Entrenamiento de 5 000 episodios para DQN y SARSA semi-gradiente.

Parámetros base utilizados:

- ▶ **Tabulares (Taxi):** ε -greedy con decaimiento $\epsilon_t = \min(1, 1000/(t+1))$, $\alpha = 0,1$, $\gamma = 1,0$.
- ▶ **Tabulares (Cliff):** ε -greedy con el mismo decaimiento, $\alpha = 0,1$, $\gamma = 0,99$.
- ▶ **DQN (base):** $\alpha = 0,0005$, $\gamma = 0,99$, buffer 50k, batch 64, target update $C = 2000$, red 64×64 , ϵ decay 0,999 (mínimo 0.01).
- ▶ **SARSA-SG (base):** $\alpha = 0,001$, $\gamma = 0,99$, red 64×64 , ϵ decay 0,999 (mínimo 0.01).

2.4.2. Métricas

Para evaluar el aprendizaje se midieron tres indicadores:

- ▶ **Recompensa media acumulada por episodio:** $\bar{R}_t = \frac{1}{t} \sum_{i=1}^t R_i$. En entornos con fuertes penalizaciones iniciales (CliffWalking) esta métrica puede permanecer muy negativa.
- ▶ **Longitud media de episodios:** número de pasos por

episodio (se reporta la media de los últimos 1000 episodios).

- **Pérdida (solo DQN)**: evolución de la loss Huber para analizar estabilidad del entrenamiento.

2.4.3. Resultados y Análisis Crítico

Taxi-v3 (control tabular)

La comparación conjunta (misma configuración de hiperparámetros) confirma que los métodos TD superan claramente a Monte Carlo en velocidad de convergencia.

Algoritmo	Recompensa media final	Longitud media final (pasos)
MC On-Policy	-138.82	113.73
MC Off-Policy	-22.32	13.69
SARSA	-5.34	13.04
Q-Learning	-4.47	13

Tabla 2.1: Resultados en Taxi-v3 (100k episodios).

Los métodos TD convergen en pocos miles de episodios, mientras que MC on-policy queda anclado en episodios largos. En Q-Learning, un ϵ fijo muy bajo (0.01) da recompensas positivas (6.31), mientras que $\gamma = 0,05$ no aprende (longitudes > 80 pasos).

En el notebook específico de Monte Carlo se obtuvieron valores distintos porque allí se usó $\epsilon = 0,3$ en off-policy (y la tabla comparativa usa $\epsilon = 0,2$), además de la variabilidad propia de la semilla: MC on-policy ≈ -156.14 con 131.85 pasos, y MC off-policy ≈ -24.74 con 13.97 pasos.

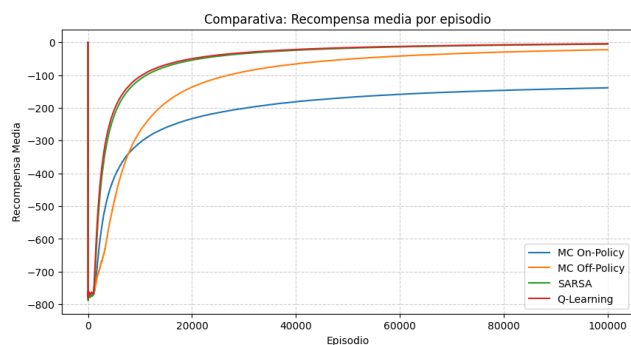


Figura 2.1: Taxi-v3: recompensa media por episodio (MC on/off, SARSA y Q-Learning).

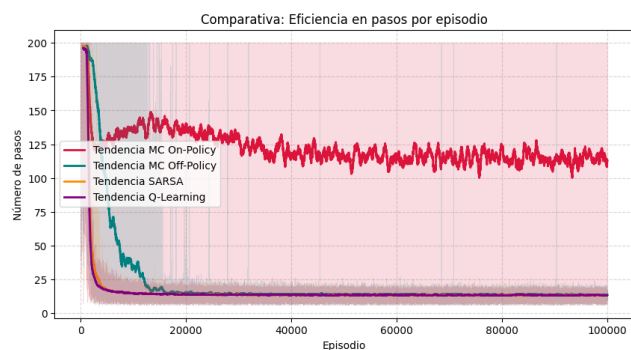


Figura 2.2: Taxi-v3: eficiencia en pasos por episodio (tendencias suavizadas).

CliffWalking-v1 (riesgo vs optimalidad)

Aquí se observa la diferencia on-policy/off-policy. Q-Learning converge a la ruta óptima (13 pasos) en el episodio greedy, mientras SARSA aprende la ruta segura (17 pasos). La media acumulada refleja muchas caídas iniciales:

- Q-Learning: recompensa media final -6845.23, longitud media 17.08 pasos.
- SARSA: recompensa media final -8025.64, longitud media 19.33 pasos.

Con $\epsilon = 0.01$ ambos mejoran notablemente: Q-Learning se acerca a 13.36 pasos y SARSA a 15.19 pasos. En Q-Learning, $\gamma = 0,05$ falla (longitud media > 180 pasos).

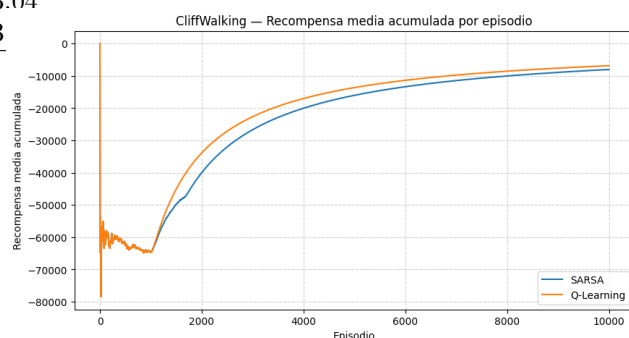


Figura 2.3: CliffWalking-v1: recompensa media acumulada por episodio (SARSA vs Q-Learning).



Figura 2.4: CliffWalking-v1: eficiencia en pasos por episodio (SARSA vs Q-Learning).

LunarLander-v3 (control aproximado)

En el entorno continuo se requiere aproximación de función. Con los hiperparámetros base, SARSA-SG alcanza recompensa superior pero DQN obtiene episodios más cortos:

- SARSA-SG (base 64x64): recompensa media final 176.38, longitud media 295.29 pasos.
- DQN (base): recompensa media final 118.11, longitud media 206.23 pasos.

La sensibilidad de DQN a sus hiperparámetros es clara: con $C = 50$ la loss explota y la recompensa cae a -493.42, mientras que $C = 10000$ mejora ligeramente (129.99). Un buffer pequeño (1000) reduce drásticamente el rendimiento (18.60).

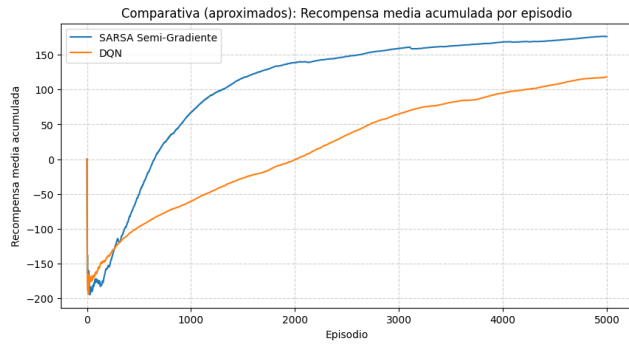


Figura 2.5: LunarLander-v3: recompensa media acumulada por episodio (SARSA-SG vs DQN).

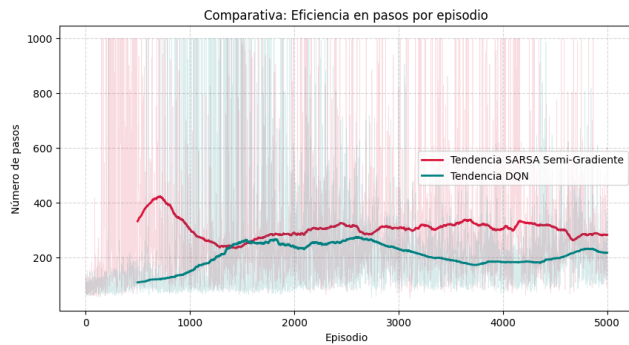


Figura 2.6: LunarLander-v3: eficiencia en pasos por episodio (SARSA-SG vs DQN).

2.5. Conclusiones

Los resultados permiten extraer varias conclusiones:

- ▶ En entornos discretos largos (Taxi-v3), los métodos TD superan ampliamente a Monte Carlo por su bootstrapping paso a paso.
- ▶ La dimensión on-policy/off-policy es crítica en CliffWalking: SARSA aprende rutas seguras y Q-Learning tiende a la ruta óptima aunque explore con riesgo.
- ▶ En entornos continuos, la aproximación con redes es imprescindible. DQN es estable pero sensible a C y al tamaño del buffer; SARSA-SG puede rendir mejor con una tasa de aprendizaje mayor.
- ▶ La comparación DQN vs SARSA-SG no es absoluta: el ranking depende de hiperparámetros y horizonte de entrenamiento.

Como trabajo futuro se propone repetir con múltiples semillas, igualar hiperparámetros entre aproximadores y evaluar variantes como Double DQN o Prioritized Replay.

Bibliografía

- [1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [2] Adam Posker. *Evaluating the potential of Deep-Q Networks for weather avoidance in aviation*. PhD thesis, 2025.
- [3] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.